LEARNING C# MADE EASY FOR BEGINNERS

C# PROGRAMMING FROM ZERO TO PROFICIENCY

(INTRODUCTION)

PATRICK FELICIA

C# From Zero to proficiency (Introduction)

A step-by-step guide to learn C#.

Table of Contents

Title Page

C# Programming from Zero to Proficiency (Introduction)

1 Introduction to C# programming

2 Thank you

Also By Patrick Felicia

Patrick Felicia

C# From Zero to Proficiency (Introduction)

Copyright © 2020 Patrick Felicia

All rights reserved. No part of this book may be reproduced, stored in retrieval systems, or transmitted in any form or by any means, without the prior written permission of the publisher (Patrick Felicia), except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either expressed or implied. Neither the author and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

First published: February 2020

Published by Patrick Felicia

Credits

Author: Patrick Felicia.

Patrick Felicia is a lecturer and researcher at Waterford Institute of Technology, where he teaches and supervises undergraduate and postgraduate students. He obtained his MSc in Multimedia Technology in 2003 and PhD in Computer Science in 2009 from University College Cork, Ireland. He has published several books and articles on the use of video games for educational purposes, including the Handbook of Research on Improving Learning and Motivation through Educational Games: Multidisciplinary Approaches (published by IGI), and Digital Games in Schools: a Handbook for Teachers, published by European Schoolnet. Patrick is also the Editor-in-chief of the International Journal of Game-Based Learning (IJGBL), and the Conference Director of the Irish Symposium on Game-Based Learning, a popular conference on games and learning organized throughout Ireland. He has also published books on coding with C# through the creation of games.

Books from the same author

JavaScript from Zero to Proficiency (Beginner): In this book, you will become more comfortable with JavaScript even if you have never coded before. This book will teach you how to code from scratch, explain programing principles such as variables, methods, statements, and much more. You will also learn how JavaScript can be used to add interaction to web pages using events, the DOM, and other key JavaScript concepts using a step-by-step method.

Unity 5 from Zero to Proficiency (Foundations): In this book, you will become more comfortable with Unity's interface and its core features by creating a project that includes both an indoor and an outdoor environment. This book only covers drag and drop features, so that you are comfortable with Unity's interface before starting to code (in the next book). After completing this book, you will be able to create outdoors environments with terrains and include water, hills, valleys, sky-boxes, use built-in controllers (First- and Third-Person controllers) to walk around the 3D environment and also add and pilot a car and an aircraft.

Support and Resources for this Book

A companion website has been set-up for this book, where you can download the full project for this book, with all the code throughout the book, so that you can check your own code, along with the code solutions for the challenges provided in each chapter.

To avail of this content, you can open the following link:

http://learncsharpforbeginners.wordpress.com/

This site also provides you with the opportunity to subscribe to a my mailing list and receive:

- your book in the pdf format (FREE copy) so that you can open it on any computer.
- regular updates and tutorials on C#.
- coupons to receive more than 30% discounts on some of my books.

This book is dedicated to Mathis & Helena

[ ]

Preface

This book is part of a series entitled **C# from Zero to Proficiency**. In this book series, you have the opportunity to learn C# programming from scratch and to become proficient in this popular programming language.

In this book entitled "**C# from Zero to Proficiency (Introduction)**" you will discover how to quickly get started with C#; you will learn about how to use variables, methods, conditional statements and you will also write your own code.

What you Need to Use this Book

To complete the project presented in this book, you only need an Integrated Development Environment such as Visual Studio Community.

In terms of computer skills, all the knowledge introduced in this book will assume no prior programming experience from the reader. Although this book includes programming, you will be guided step-by-step. So for now, you only need to be able to perform common computer tasks such as typing, and opening and saving files.

Who this Book is for

If you can answer **yes** to all these questions, then this book is for you:

- Are you a total beginner in C# programming?
- Would you like to get started fast with C# concepts?
- Although you may have had some prior exposure to C#, would you like to delve more into C#?

If you can answer yes to all these questions, then this book is **not** for you:

- Can you already easily code in C#?
- Are you looking for a reference book on C#?
- Are you an experienced (or at least advanced) C# programmer?

If you can answer yes to these three questions, you may instead look for other books in this series. To see the content and topics covered by these books, you can check the official website: **http://www.learncsharpforbeginners.wordpress.com**.

Improving the Book

Although great care was taken in checking the content of this book, I am human, and some errors could remain in the book. As a result, it would be great if you could let me know of any issue or error you may have come across in this book, so that it can be solved and the book updated accordingly. To report an error, you can email me (learncsharpforbeginners@gmail.com) with the following information:

- Name of the book.
- The page where the error was detected.
- Describe the error and also what you think the correction should be.

Once your email is received, the error will be checked, and, in the case of a valid error, it will be corrected and the book page will be updated to reflect the changes accordingly.

Supporting the Author

A lot of work has gone into this book and it is the fruit of long hours of preparation, brainstorming, and finally writing. As a result, I would ask that you do not distribute any illegal copies of this book.

If some of your friends are interested in the book, you can refer them to the book's official website (**http://www.learncsharpforbeginners.wordpress.com/**) where they can either buy the book, enter a monthly draw to be in for a chance of receiving a free copy of the book, or to be notified of future promotional offers.

Table of Contents

1 Introduction to C# programming

What is C#
Choosing a code editor
Installing Visual Studio
Creating your first application
Introduction to C# syntax
Statements
Comments
Variables
Using variables
Operators
Conditional statements
Methods
Scope of variables
A few things to remember when you create your applications (checklist)
Best practices
Level roundup

2 Thank you

Content Covered by this Book

- Chapter 1, *Introduction to C# programming*, provides an introduction to general programming concepts. including C# syntax, variables and methods. In this chapter, you will start to create your first application and combine variables, and methods.
- Chapter 2 summarizes the topics covered in this book and provides you with more information on the next steps to follow.

What you Need to Use this Book

To complete the project presented in this book, you only need Visual Studio (e.g., Visual Studio Community). In terms of computer skills, all knowledge introduced in this book will assume no prior programming experience from the reader. So for now, you only need to be able to perform common computer tasks, such as downloading items, opening and saving files, be comfortable with dragging and dropping items and typing.

Who this Book is for

If you can answer **yes** to all these questions, then this book is for you:

1. Would you like to learn how to code in C#?
2. Would you like to know how to use C# to create simple command-line programs?
3. Would you like to discover more C# features?
4. Although you may have had some prior exposure to C#, would you like to delve more into Object-Oriented Programming?

Who this Book is not for

If you can answer yes to all these questions, then this book is **not** for you:

1. Can you already code your own C# code?
2. Are you looking for a reference book on C# programming?
3. Are you a professional C# developer?

If you can answer yes to all three questions, you may instead look for the other books in the series on the official website (http://www.learncsharpforbeginners.wordpress.com).

How you will Learn from this Book

Because all students learn differently and have different expectations of a course, this book is designed to ensure that all readers find a learning mode that suits them. Therefore, it includes the following:

- A list of the learning objectives at the start of each chapter so that readers have a snapshot of the skills that will be covered.
- Each section includes an overview of the activities covered.
- Many of the activities are step-by-step, and learners are also given the opportunity to engage in deeper learning and problem-solving skills through the challenges offered at the end of each chapter.
- Each chapter ends-up with a quiz and challenges through which you can put your skills (and knowledge acquired) to the test. Challenges consist in coding, debugging, or creating new features based on the knowledge that you have acquired in the chapter.
- The book focuses on the core skills that you need. While some sections go into more detail, once concepts have been explained, links are provided to additional resources, where necessary.
- The code is introduced progressively and it is also explained in detail.
- You also gain access to several videos that help you along the way, especially for the most challenging topics.

Format of each Chapter and Writing Conventions

Throughout this book, and to make reading and learning easier, text formatting and icons will be used to highlight parts of the information provided and to make the book easy to read.

Special Notes

Each chapter includes resource sections, so that you can further your understanding and mastery of C#; these include:

- A quiz for each chapter: these quizzes usually include 10 questions that test your knowledge of the topics covered throughout the chapter. The solutions are provided on the companion website.
- A checklist: it consists of between 5 and 10 key concepts and skills that you need to be comfortable with before progressing to the next chapter.
- Challenges: each chapter includes a challenge section where you are asked to combine your skills to solve a particular problem.

Author's notes appear as illustrated below:

Author's suggestions appear in this box.

Code appears as illustrated below:

```
public int score;
public string playersName = "Sam";
```

Checklists that include the important points covered in the chapter appear as illustrated below:

- Item1 for check list.
- Item2 for check list.
- Item3 for check list.

How Can You Learn Best from this Book?

- Talk to your friends about what you are doing.

We often think that we understand a topic until we have to explain it to friends and answer their questions. By explaining your different projects, what you just learned will become clearer to you.

- Do the exercises.

All chapters include exercises that will help you to learn by doing. In other words, by completing these exercises, you will be able to better understand the topics and gain practical skills (i.e., rather than just reading).

- Don’t be afraid of making mistakes.

I usually tell my students that making mistakes is part of the learning process; the more mistakes you make and the more opportunities you have for learning. At the start, you may find the errors challenging, or you may find that your application does not work as expected until you understand what went wrong.

- Challenge yourself.

All chapters include a challenge section where you can decide to take on a particular challenge to improve your game or skills. These challenges are there for you to think creatively and to apply the knowledge that you have acquired in each chapter using a problem-based approach.

- Learn in chunks.

It may be challenging to go through five or six chapters straight, as it may lower your motivation. Instead, give yourself enough time to learn, go at your own pace, and learn in small units (e.g., between 15 and 20 minutes per day). This will do at least two things for you: it will give your brain the time to "digest" the information that you have just learned, so that you can start fresh the following day. It will also make sure that you don’t "burn-out" and that you keep your motivation levels high.

Feedback

While I have done everything possible to produce a book of high quality and value, I always appreciate feedback from readers so that the book can be improved accordingly. If you would like to give feedback on this book, you can email me at **learncsharpforbeginners@gmail.com**.

Improving the Book

Although great care was taken in checking the content of this book, I am human, and some errors could remain in the book. As a result, it would be great if you could let me know of any issue or error you may have come across in this book, so that it can be solved and so that the book can be updated accordingly. To report an error, you can email me (**learncsharpforbeginners@gmail.com**) with the following information:

- Name of the book.
- The page or section where the error was detected.
- Describe the error and also what you think the correction should be.

Once your email is received, the error will be checked, and, in the case of a valid error, it will be corrected, and the book will be updated to reflect the changes accordingly.

Supporting the Author

A lot of work has gone into this book and it is the fruit of long hours of preparation, brainstorming, and finally writing. As a result, I would ask that you do not distribute any illegal copies of this book.

This means that if a friend wants a copy of this book, s/he will have to buy it through the official channels or the book's official website:

http://www.learncsharpforbeginners.wordpress.com.

If some of your friends are interested in the book, you can refer them to the book's official website **http://www.learncsharpforbeginners.wordpress.com** where they can either buy the book, or join the mailing list to be notified of future promotional offers or enter a monthly draw and be in for a chance to receive a free copy of the book.

[ ]

1
Introduction to C# programming

In this section, we will go through an introduction to C# programming and look at key aspects that you will need for your games, including:

- C# syntax.
- Variable types and scope.
- Methods.

So, after completing this chapter, you will be able to:

- Understand key concepts related to C# programming.
- Understand the concepts of variables and methods.
- Create and use your own methods.

The code solutions for this chapter are included in the **resource pack** that you can download by following the instructions included in the section entitled "**Support and Resources for this Book**".

What is C#

C# is an object-oriented language developed by Microsoft and has been used, in many cases, to develop desktop applications. As it is part the .NET framework, it is also used for web development and apps. The fact that it is an object-oriented programming language means that a C# programme consists of a collection of objects that interact with each other.

C# is used for many applications, including the creation of games, especially with the software Unity.

Choosing a code editor

In the next sections, you will start to write your own C# code. For this purpose, you will need to use a code editor, and more specifically, an Integrated Development Environment (IDE). These IDEs include all the necessary tools to compile and execute your code.

Using an Ide will provide you with an easier way to create, organize and check your code, with some of the following features:

- **Text highlighting**: so that you code is clearer to read and to modify.
- **Auto-completion**: so that you don't have to remember the name of all the variables in your code, or the name of some common methods.
- **Error highlighting**: so that some obvious errors are highlighted before you even test your code.

There are many free IDEs available with their advantages and drawbacks, including:

- Visual Studio Code.
- Visual Studio Community.
- MonoDevelop.
- NetBeans.
- #Develop.
- Rider.

All these IDEs considered, we will be using Visual Studio Community for this book (although you could use any of the other IDEs if you wish).

You can download Visual Studio Community from the following site: https://visualstudio.microsoft.com/free-developer-offers/

Installing Visual Studio

Please download Visual Studio from the official site and follow the instructions; you should be able to download and install a version for Windows or Mac OS from the following site:

https://visualstudio.microsoft.com/free-developer-offers/

Creating your first application

After this brief introduction, it is now your turn to code.

So that you get to experiment with your C# code, please do the following after installing Visual Studio:

- Please launch VS (Visual Studio).
- From the main window, select the button labelled **"New Project"**, as illustrated in the next figure.

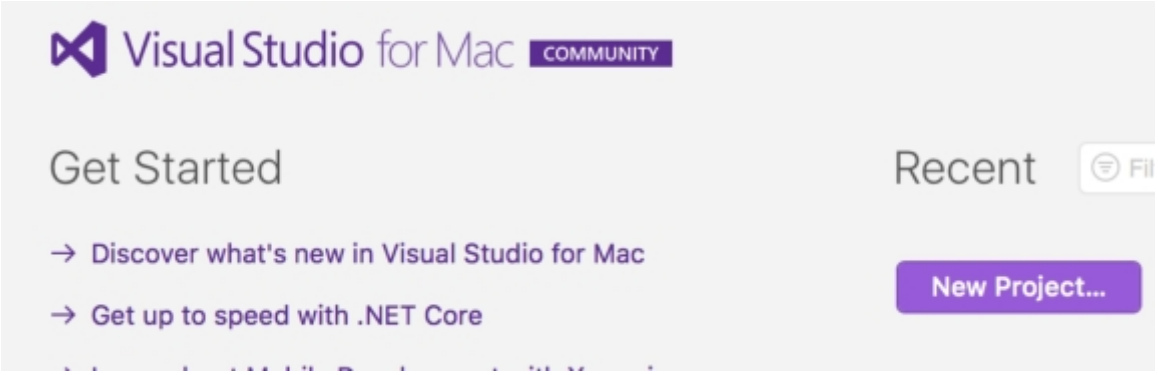

Figure 1-1: Creating your project

- In the new window, select the option **Other > .NET > Console Project (C#)**, as illustrated in the next figure:

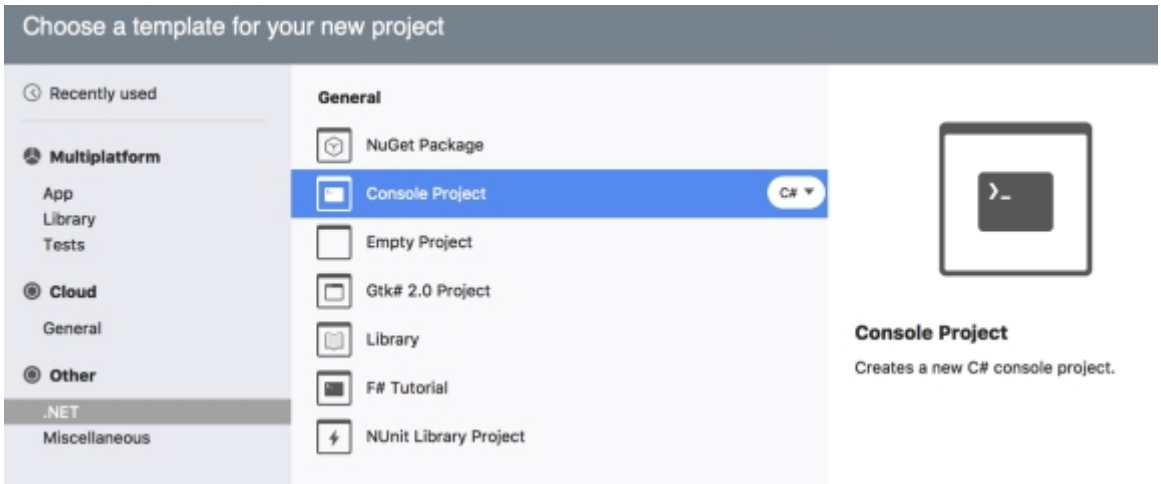

Figure 1-2: Creating a Console Project

- By choosing this option, you specify that you will build a project that will be executed in the console window.
- Please click on the button labelled **"Next"**.
- In the next window, enter the name and location of your new project (for example **myFirstproject**), as illustrated in the next figure.

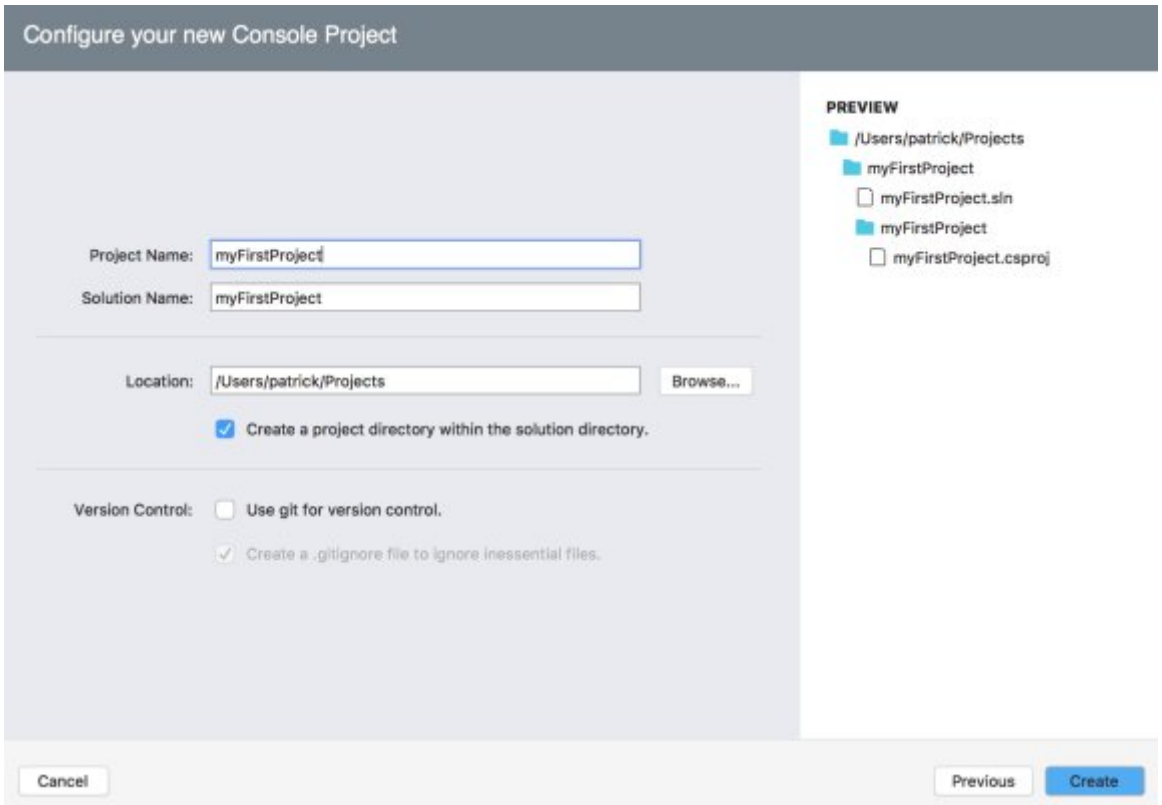

Figure 1-3: Setting-up your Project

- Click on the button labelled "**Create**".

Once this is done Visual Studio (VS) will create all the necessary folders for your project and the code editor should open as follows.

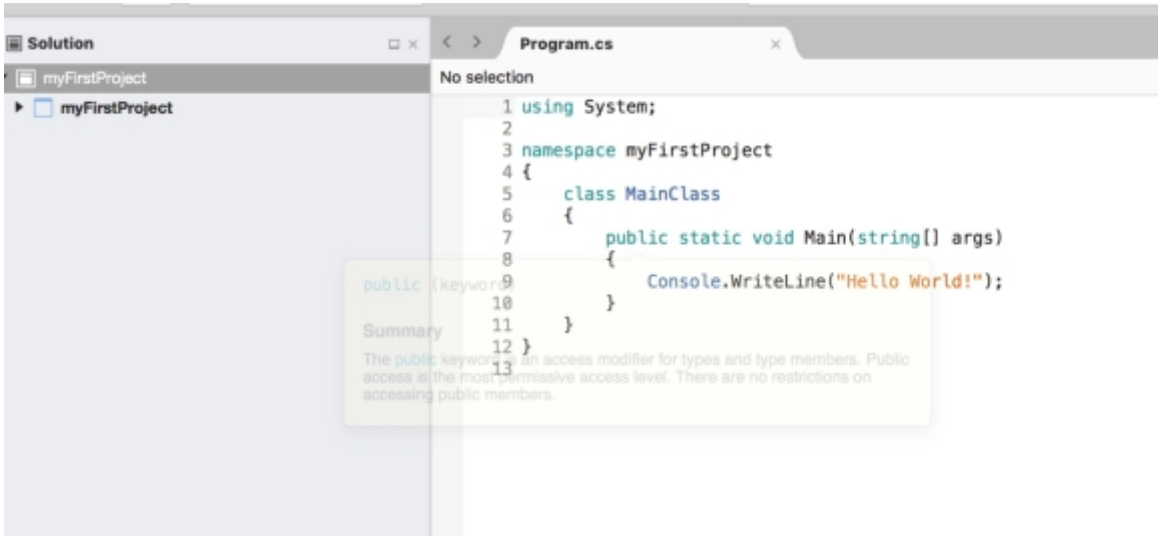

Figure 1-4: Your first code

By default, you will see that some code has already been created for you, as follows:

```
using System;
namespace myFirstProject
{
```

```
class MainClass
{
public static void Main(string[] args)
{
Console.WriteLine("Hello World!");
}
}
}
```

In the previous code:

- We declare that we will be using the library (or path) called **System**. This is so that we can employ some of the useful files (i.e., classes) included in this namespace (or library).
- We declare a namespace called **myFirstProject**, and we will see this concept later in the book. This is the equivalent of a path or library for some of the classes that you will use in your application.
- We declare a class called **MainClass**. As we will see later, your application will be made of a collection of classes. One of these classes will be the class **MainClass** which the starting point for your application.
- We also declare a method called **Main**. This method will be the starting point for our application.
- In this method, and this is the most important part for now, we write the text "**Hello World"** in the console window.

You can now compile and execute this code by pressing the black triangle located in the top-left corner of the window.

Figure 1-5: Running the project

You can also proceed as follows to run your application:

- From the top menu, select: **Run > Run With > Custom Configuration...**This will open a window that looks like the following figure.

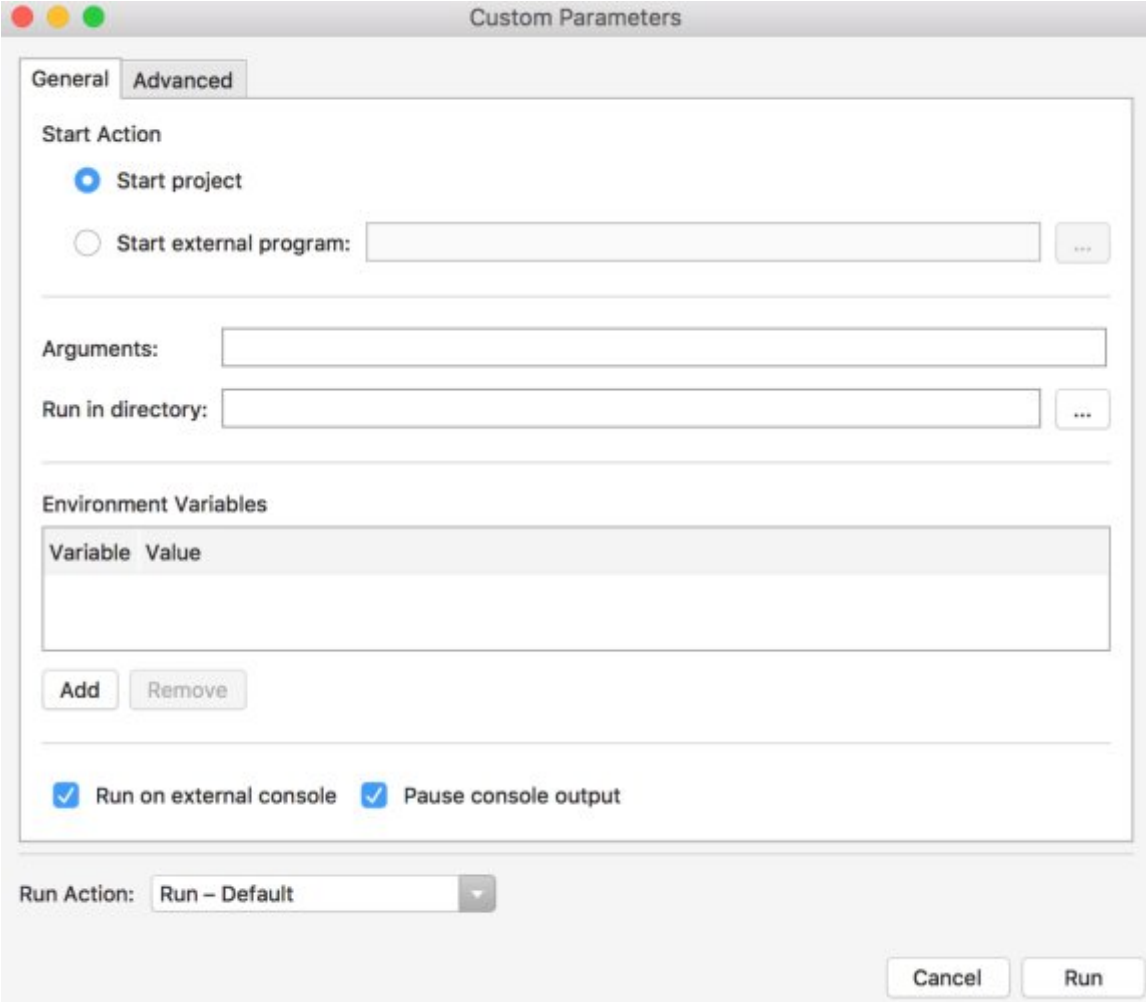

Figure 1-6: Running your project with custom settings

- Once the window is open, please leave all the default options as they are and then press the button labelled **Run**.
- Once this is done, the console window should appear as follows.

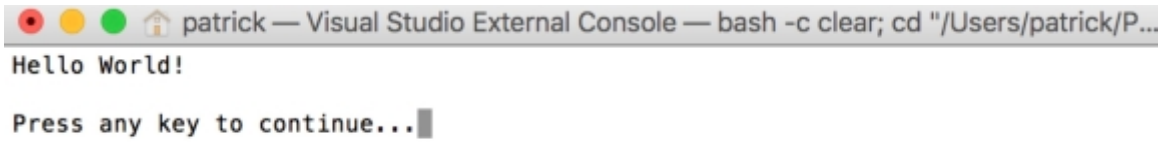

Figure 1-7: Your first console application

As illustrated in the previous figure, the text **"Hello World"** was written and displayed in the console window, based on your code.

We could now change our code so that the message displayed is different:

- Please close the console window.
- Back in VS, please edit your code and modify it as follows (new code in bold).

```
public static void Main(string[] args)
{
Console.WriteLine("Hello, this is my fist line of code");
}
```

In the previous code, we just modified the message displayed in the console window.

- You can then now run your code and see the new message, as follows:

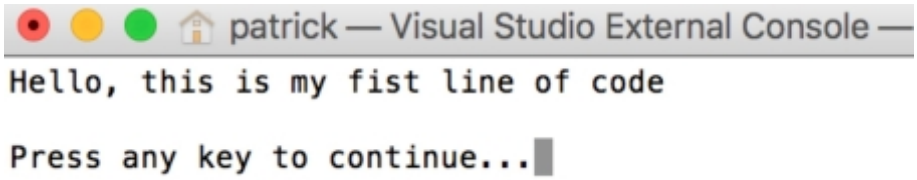

Figure 1-8: Displaying your own message

Introduction to C# syntax

When you are using C#, you are communicating with the system asking it to perform actions. To communicate with the system, you are using a language or a set of words bound by a syntax that the computer and you know. This language consists of keywords, key phrases, and a syntax that ensures that the instructions are written and (more importantly) understood properly.

In computer science, this language needs to be exact, precise, unambiguous, and with a correct syntax. In other words, it needs to be **exact**.

When writing C# code, you will be following a syntax; this syntax will consist of a set of rules that will make it possible for you to communicate your instructions unambiguously. In addition to its syntax, C# also uses classes, and your C# applications will, by default, be saved as classes.

In the next section, we will learn how to use this syntax. If you have already coded in JavaScript or C#, some of the information provided in the rest of this chapter may look familiar and this prior exposure to C# will definitely help you. This being said, C# and JavaScript, despite some relative similarities, are quite different in many aspects; for example, in C#, variables and methods are declared differently.

When programming in C#, you will be using a specific syntax to communicate with the system; this syntax will be made of sentences that will be used to convey information on what you would like the computer to do; these sentences or statements will include a combination of keywords, variables, methods, or events; and the next section will explain how you can confidently build these sentences together and consequently program in C#.

Statements

When you code in C#, you need to tell the system to execute your instructions (e.g., print information) using statements. A statement is literally an order or something that you ask the system to do. For example, in the next line of code, the statement will tell the system to print a message in the **Console** window:

```
Console.WriteLine("Hello Word");
```

When writing statements, you will need to follow several rules, including the following:

- **The order of statements**: each statement is executed in the same order as it appears in the application. For example, in the next example, the code will print **hello**, then **world;** this is because the associated statements are in that particular sequence.

```
Console.WriteLine("hello");
Console.WriteLine("world");
```

- **Statements are separated by semi-colons** (i.e., semi-colon at the end of each statement).

Note that several statements can be added on the same line, as long as they are separated by a semi-colon.

For example, the next line of code has a correct syntax, as all of its statements are separated by a semi-colon.

```
Console.WriteLine("hello"); Console.WriteLine("world");
```

- **Multiple spaces are ignored for statements**; however, it is good practice to add spaces around the operators +, **-**, /, or **%** for clarity. For example, in the next code snippet, we say that **a** is equal to **b**. You may notice that spaces have been included both before and after the operator =.

```
a = b;
```

- **Statements to be executed together (e.g., based on the same condition) can be grouped using code blocks**. In C#, code blocks are symbolized by curly brackets (e.g., **{** or **}**). So, in other words, if you needed to group several statements, you would include all of them within the same set of curly brackets, as follows:

```
{
Console.WriteLine("hello stranger!");
Console.WriteLine("today, we will learn about programming");
```

}

As we have seen earlier, a statement usually employs or starts with a **keyword** (i.e., a word that the computer knows). Each of these keywords has a specific purpose, and common ones are as follows:

- Printing a message in the **console** window: the keyword is **Console.WriteLine**.
- Declaring a variable: the keyword, in this case, depends on the type of the variable that is declared (e.g., **int** for integers, **string** for text, or **bool** for Boolean variables), and we will see more about these in the next sections.
- Declaring a method: the keyword to be used depends on the type of the data returned by the method. For example, in C#, the name of a method is preceded by the keyword **int** when the method returns an **integer**; it is preceded by the keyword **string** when the method returns a **string**, or by the keyword **void** when the method does not return any information.
- Marking a block of instructions to be executed based on a condition: the keywords are **if** and **else**.
- Exiting a method: the keyword is **return**.

Now that you know a bit more about statements, let's modify your initial code to include more statements:

- Please open the file **Program.cs** that we have used earlier in VS.
- Add the following code (new code in bold) to it:

```
Console.WriteLine("Hello, this is my fist line of code");
{
Console.WriteLine("this is my first application");
Console.WriteLine("This statement and the previous one belong to the same
block of instructions");
}
```

In the previous code:

- We have created a block of instructions that starts with an opening curly bracket and that ends with a closing curly bracket.
- Within this block, we have created two statements based on alert boxes; both statements are ending with a semi-colon.

You can now save your code and run the application; you should see two additional messages displayed in the console window (i.e., the ones that you have just added).

For now, using blocks of instructions may look like it will not make much of a difference to your coding, and this is true for now. This being said, blocks of

instructions are very helpful, as we will see later, especially when we need to tell the browser to execute a list of instructions. This is because, in this case, instead of telling the system to execute each instruction individually, we can instead, ask for a block of instructions to be executed.

Comments

In C#, you can use comments to explain your code and to make it more readable by others. This becomes important as the size of your code increases; and it is also important if you work in a team, so that other team members can understand your code and make amendments in the right places, if and when it is needed.

Code that is commented is usually not executed. There are two ways to comment your code in C# using either **single-** or **multi-line** comments.

In single-line comments, a **double forward slash** is added at the start of a line or after a statement, so that this line (or part thereof) is commented, as illustrated in the next code snippet.

```
//the next line prints Hello in the console window
Console.WriteLine("Hello");
//the next line declares the variable name
string name;
name = "Hello";//sets the value of the variable name
```

In multi-line comments, any code between the characters forward slash and star " **/*"** and the characters star and forward slash **"*/"** will be commented, and this code will not be executed. This is also referred as **comment blocks**.

```
/* the next lines after the comments will print the message "hello" in the console window
we then declare the variable name and assign a value
*/
Console.WriteLine("Hello");
string name;
name = "Hello";//sets the value of the variable name
//Console.WriteLine("Hello World")
/*
string name;
name = "My Name";
*/
```

In addition to providing explanations about your code, you can also use comments to prevent part of your code to be executed. This is very useful when you would like to debug your code and find where the error or bug might be, using a very simple method. By commenting sections of your code, and by using a process of elimination, you can usually find the issue quickly. For example, you can comment all the code and run the application; then comment half the code, and run the application. If it works without no errors, it means that the error is within the code that has been commented, and if it does not work, it means that the error is in the code that has not been commented. In the first case (when the code works), we could then just comment half of the portion of the code that has already been commented. So, by successively commenting more specific areas

of our code, we can get to discover what part of the code includes the bug. This process is often called **dichotomy** (as we successively divide a code section into two).

Let's use comments in our application:

- Please open the file **Program.cs**.
- Modify the code as follows (new code in bold).

```
public static void Main(string[] args)
{
//Console.WriteLine("Hello, this is my fist line of code");
Console.WriteLine("Hello, this is my second line of code");
{
Console.WriteLine("this is my first application");
```

In the previous code, we comment the message **"Hello this is my first line of code"** and replace it with the message **"Hello this is my second line of code"**.

Now that you have made these changes, please save your code and run the application; you should see that, this time, the message **Hello this is my first line of code** has been replaced with **"Hello this is my second line of code"**.

We could also use multi-line comments as follows (new code in bold).

```
Console.WriteLine("Hello, this is my second line of code");
{
/*Console.WriteLine("this is my first application");
Console.WriteLine("This statement and the previous one belong to the same
block of instructions");*/
}
```

In the previous code, we used a multi-line comment. This means that the two lines comprised between /* and */, while they belong to the file, will not be executed.

If you save your code and run the application, you should see that the only message displayed onscreen is **"Hello this is my second line of code"**.

Variables

Now that you know a bit more about statements and comments, we will start to learn more about variables, which are quite important in C#, as you will probably use them in most of your programs to perform calculations or to work with text.

So what is a variable?

A variable can be compared to a container that includes a value that may change over time. When using variables, we usually need to: (1) declare the variable by specifying its type, (2) assign a value to this variable, and (3) possibly combine this variable with other variables using operators, as illustrated in the next code snippet.

```
int myAge;//we declare the variable myAge
myAge = 20;// we set the variable myAge to 20
myAge = myAge + 1; //we add 1 to the variable myAge
```

In the previous example, we have declared a variable called **myAge** and its type is **int** (as in **integer**). We save the value **20** in this variable, and we then add **1** to it.

Note that in C# the variable is declared using its type followed by its name. As we will see later, we will also need to use what is called an **access modifier** in order to specify how and from where this variable can be accessed.

Also note that in the previous code, we have assigned the value **myAge + 1** to the variable **myAge**; the = operator is an assignment operator; in other words, it is there to assign a value to a variable and is not to be understood in a strict algebraic sense (i.e., that the values or variables on both sides of the = sign are equal).

To make C# coding easier and leaner, you can declare several variables of the same type in one statement. For example, in the next code snippet, we declare three variables **v1**, **v2**, and **v3** in one statement. This is because they are of the same type (i.e., they are **integers**).

```
int v1,v2,v3;
int v4=4, v5=5, v6=6;
```

In the code above, the first line declares the variables **v1**, **v2**, and **v3**. All three variables are **integers**. In the second line of code, not only do we declare three variables simultaneously, but we also initialize them by setting a value for each of these variables.

When using variables, there are a few things that we need to determine including their name, their type and their scope:

- **Name of a variable:** a variable is usually given a unique name so that it can be identified easily and uniquely. The name of a variable is usually referred to as an **identifier**. When defining an identifier, it can contain letters, digits, a minus, an underscore or a dollar sign, and it usually begins with a letter. Identifiers cannot be keywords (for example the keyword **if** cannot be used as an identifier).

- **Type of variable:** variables can hold several types of data, including numbers (e.g., integers, doubles or floats), text (e.g., strings or characters), Boolean values (e.g., true or false), arrays, or objects (we will see the concept of arrays later in this chapter).

```
string myName = "Patrick";//the text is declared using double quotes
int currentYear = 2017;//the year needs no decimals and is declared as an integer
float width = 100.45f;//the width is declared as a float (i.e., with decimals)
```

- **Variable declaration:** variables need to be declared so that the system knows what you are referring to if you use this variable in your code. The first step in using a variable is to declare or define this variable. At the declaration stage, the variable does not have to be assigned a value, as this can be done later. In the next example, we declare a variable called **myName** and then assign the value **"My Name"** to it.

```
string myName;
myName = "My Name"
```

- **Scope of a variable:** a variable can be accessed in specific contexts that depend on where in the application the variable was initially declared. We will look at this concept later.
- **Accessibility level:** as we will see later, a C# program consists of classes; for each of these classes, the methods and variables within can be accessed depending on their **accessibility** levels and we will look at this principle later.

Common variable types include:

- **String**: same as text.
- **Int**: integer (1, 2, 3, etc.).
- **Boolean**: true or false.
- **Float**: with a fractional value (e.g., 1.2f, 3.4f, etc.).
- **Arrays**: a group of variables of the same type. If this is unclear, not to worry, this concept will be explained further in this chapter.

Using variables

In this section, we will start to use variables and combine them using statements and operators. So let's get started:

- Please add the following code just after the code that you have previously created, as follows (new code in bold).

//Console.WriteLine("Hello, this is my second line of code");
{
/*Console.WriteLine("this is my first application");
Console.WriteLine("This statement and the previous one belong to the same block of instructions");*/
}
int dateOfBirth = 1990;
int currentYear = 2017;
int age = currentYear - dateOfBirth;
Console.WriteLine("My age is " + age);
In the previous code:

- We have commented the previous code that displayed **"Hello, this is my second line of code"** using the two forward slashes; so this code will not be executed.
- We declare two variables **dateOfBirth** and **currentYear**.
- We then declare a variable called **age**.
- We perform a subtraction between the variable **currentYear** and the variable **dateOfBirth**, and we then save the result in the variable called **age**.
- Finally, we print the text "**My age is** ", followed by the value of the variable called **age**.

Note that adding a number and a string is equivalent to concatenating these two, so the result of this addition will be a string.

You can now save your code and run your application; you should see the message "**My age is 27**" in the **Console** window.

Operators

Once we have declared and assigned a value to a variable, we can use operators to modify or combine this variable with other variables. There are different types of operators available in C#, including arithmetic operators, assignment operators, comparison operators and logical operators.

So let's look at each of these.

- **Arithmetic operators** are used to perform arithmetic operations including additions, subtractions, multiplications, or divisions. Common arithmetic operators include **+**, **-**, *****, **/**, or **%** (modulo) and their use is illustrated in the next code snippet.

```
int number1 = 1;// the variable number1 is declared
int number2 = 1;// the variable number2 is declared
int sum = number1 + number2;// adding two numbers and store them in sum
int sub = number1 - number2;// subtracting two numbers and store them in sub
```

- **Assignment operators** can be used to assign a value to a variable and include **=**, **+=**, **-=**, ***=**, **/=** or **%=** and their use is illustrated in the next code snippet.

```
int number1 = 1;
int number2 = 1;
number1+=1; //same as number1 = number1 + 1;
number1-=1; //same as number1 = number1 - 1;
number1*=1; //same as number1 = number1 * 1;
number1/=1; //same as number1 = number1 / 1;
number1%=1; //same as number1 = number1 % 1;
```

Note that the **=** operator, when used with strings, will concatenate them (i.e., add them one after the other to create a new string). When used with a number and a string, the same will apply and the result is a string (for example "**Hello"+1** will result in "**Hello1**").

- **Comparison operators** are often used for conditions to compare two values. Comparison operators include **==**, **!=**, **>**, **<**, **>=** and **>=**, and their use is illustrated in the next code snippet.

```
if (number1 == number2); //if number1 equals number2
if (number1 != number2); //if number1 and number2 have different values
if (number1 > number2); //if number1 is greater than number2
if (number1 >= number2); //if number1 is greater than or equal to number2
if (number1 < number2); //if number1 is less than number2
```

```
if (number1 <= number2); //if number1 is less than or equal to number2
```

Conditional statements

While we have looked at simple variables such as numbers or text, and how to modify them through operators, we will now look at conditional statements.

Conditional statements are statements that are performed based on a condition, hence their name. Their syntax is usually as follows:

if (condition) statement;

This means **if the condition is verified (or true) then (and only then) the statement is executed**. When we assess a condition, we test whether a declaration (or statement) is true.

For example, by typing **if (a == b)**, we mean **"if it is true that a equals b"**. Similarly, if we type **if (a>=b)** we mean **"if it is true that a equals or is greater than b"**

As we will see later, we can also combine conditions. For example, we can decide to perform a statement if two (or more) conditions are true. For example, by typing **if (a == b && c == 2)** we mean **"if a equals b and c equals 2"**. In this case using the operator **&&** means **AND**, and that both conditions will need to be true. We could compare this to making a decision on whether we will go sailing tomorrow.

For example, we could translate the sentence "**if the weather is sunny and the wind speed is less than 5km/h then I will go sailing**" as follows.

if (weatherIsSunny == true && windSpeed < 5) IGoSailing = true;

When creating conditions, as for most natural languages, we can use the operator **OR** noted **||**. Taking the previous example, we could translate the following sentence "**if the weather is too hot or the wind is faster than 5km/h then I will not go sailing** " as follows.

if (weatherIsTooHot == true || windSpeed > 5) IGoSailing = false;

Another example could be as follows.

```
if (myName == "Patrick") alert("Hello Patrick");
else alert ("Hello Stranger");
```

In the previous code, we display the text "**Hello Patrick**" if the variable **myName** equals to "**Patrick**"; otherwise, we display the message "**Hello Stranger**".

When we assess true and false conditions, we are effectively applying what is called **Boolean logic**. Boolean logic deals with Boolean numbers that have two values 1 and 0 (or true and false). By evaluating conditions, we are effectively processing Boolean numbers and applying Boolean logic. While you don't need to know about Boolean logic in depth, some operators for Boolean logic are important, including the **!** operator. It means **NOT** or "**the opposite of**". This means that if a variable is true, its opposite will be false, and vice versa. For example, if we consider the variable **weatherIsGood = true**, the value of **!weatherIsGood** will be **false** (its opposite). So the condition **if (weatherIdGood == false)** could also be written **if (!weatherIsGood)** which would literally translate as "if the weather is **NOT** good".

Let's experiment with conditional statements:

- Please open the file **Program.cs**.
- Add the following code (new code in bold) at the end of the **Main** method:

```
Console.WriteLine("My age is " + age);
Console.WriteLine("Please enter your name: ");
string nameOfUser = Console.ReadLine();
if (nameOfUser == "Patrick") Console.WriteLine("Hello Patrick");
else Console.WriteLine("Hello Stranger");
```

In the previous code:

- We write a message that asks the user to enter his/her name.
- We declare the variable **nameOfUser**. This variable will then contain the answer (a string) entered by the user.
- We store the name using the method **Console.ReadLine** method, which is a method provided by C# to read text entered by the user (after the user has pressed the **Enter**).
- Once the name has been entered by the user, it is stored in the variable **nameOfUser**.
- We then execute a conditional statement based on the value of the variable **nameOfUser**. If the text entered is "**Patrick**" (note that this is case-sensitive, so the browser is looking for this exact string, accounting for the case of each letter within), then the message "**Hello Patrick**" is displayed. Otherwise, the message "**Hello Stranger**" is displayed.

You can now save your code and run the application, and you should see the following window:

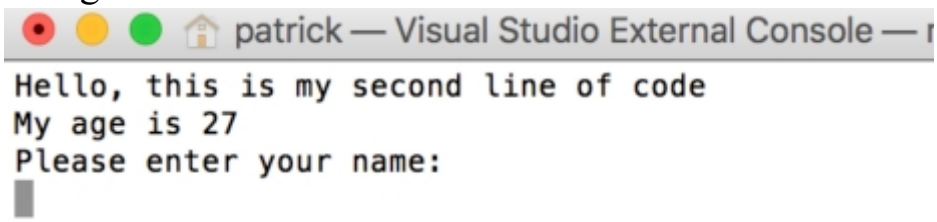

Figure 1-9: Recording text entered by the user

- As you enter the name **"Patrick"** and press the button labelled "**OK**", the message **"Hello Patrick"** should appear.

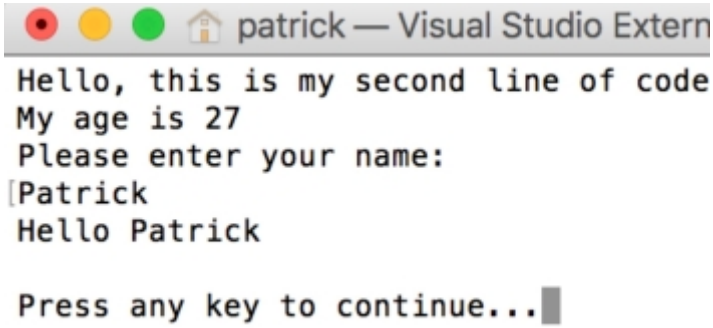

Figure 1-10: Displaying a message based on the user input

- You can then close the **console** window.

Now that we have used conditional statements for strings, we can also experiment with numbers. So, please amend the code in the file **Program.cs** as follows (new code in bold).

```
/*int dateOfBirth = 1990;
int currentYear = 2017;
int age = currentYear - dateOfBirth;
Console.WriteLine("My age is " + age);
Console.WriteLine("Please enter your name: ");
string nameOfUser = Console.ReadLine();
if (nameOfUser == "Patrick") Console.WriteLine("Hello Patrick");
else Console.WriteLine("Hello Stranger");*/
Console.WriteLine("What is 1 + 1");
int result = Console.ReadLine();
if (result == "2") Console.WriteLine("Well done");
else Console.WriteLine("Sorry, wrong answer");
```

In the previous code:

- We comment the previous code.
- We display text that invites the user to write the solution to the question **"What is 1 + 1"**.
- We declare a variable called **result**.
- We then ask the user to enter his/her guess using the **Console.ReadLine** method.
- Once the answer has been entered, it is stored in the variable **result**.
- We then perform a conditional statement based on the value of the variable **result**. If the text entered is "**2**", then the message "**Well done**" is displayed, otherwise, the message "**Sorry Wrong Answer**" is displayed instead.

Note that in the previous example, we ask for a number to be entered; however, because it is entered using **Console.ReadLine**, it will be considered as a string by C#.

This is the reason why we are using double quotes around the number "**2**" when testing the answer; ideally, and we will look into this later, we could have converted the entry to a number and the code could have looked like the following:

```
Console.WriteLine("What is 1 + 1");
string resultAsString = Console.ReadLine();
int result = Int32.Parse(resultAsString);
if (result == 2) Console.WriteLine("Well done");
```

In the previous code:

- We declare the variable **resultAsAString**.
- We convert the answer provided by the user, which is a string so far, to an integer (i.e., a number with no decimals).
- This new number is then stored in the variable called **result**.
- The conditional statement now uses the number **2** as a condition as opposed to the string "**2**".

Note that if you want to copy/paste the code from this book, make sure that the double quotes are straight and not curled, otherwise, the system may perceive the curled quotes as an error.

You can now save your code and run your application, and you should see, as for the previous example, a text asking you to enter your answer to the question "What is 2 + 2".

- After entering the number **2** and pressing the "**OK**" button, a message saying **"Well done"** should appear.

Methods

Now that you can declare and combine numbers and strings, it is a good time to start looking into methods.

Methods can be compared to a friend or a colleague to whom you gently ask to perform a task, based on specific instructions, and to return information afterwards, if need be.

For example, you could ask your friend the following: "**Can you please tell me when I will be celebrating my 20th birthday given that I was born in 2000**". So you give your friend (who is good at Math :-)) the information (i.e., the date of birth) and s/he will calculate the year of your 20th birthday and give this information back to you. So in other words, your friend will be given an input (i.e., the date of birth) and s/he will return an output (i.e., the year of your 20th birthday). Methods work exactly this way: they are given information (and sometimes not), perform an action, and then sometimes, if needed, return information.

In programming terms, a method is a block of instructions that performs a set of actions. It is executed when invoked (or put more simply **called**) from the application, or when an event occurs (e.g., when the player has clicked on a button; we will see more about events in the next section). As with variables, methods are declared and can also be called.

Methods are very useful because once the code for a method has been created, it can be called several times without the need to rewrite the same code all over again. Also, because methods can take parameters, a method can process these parameters and produce or return information accordingly; in other words, they can perform different actions and produce different outputs based on the input. As a result, methods can do one or all of the following:

- Perform an action.
- Return a result.
- Take parameters and process them.

A method is usually declared using two keywords followed by its name as follows:

```
accessType typeOfDataReturned nameOfTheFunction (listOfParameters)
{
Perform actions here
}
```

In the previous snippet:

- The access type defines the scope of the method or, put simply, from where this method will be available (or can be called).
- The type of the data returned can be: void (no data is returned) or any type of data that this method could return such as integers, Boolean, or strings.

- The previous information is then followed by the name of the method.
- The list of parameters can be empty, if this method does not take any parameter. Otherwise, it includes a list of parameters preceded by their type and separated by a comma; we will cover this aspect later.

Using this template, we could, for example, define the following method:

```
public void myMethod ()
{
Console.WriteLine("hello");
}
```

In the previous code:

- The method is declared as public.
- Its return type is **void**, which means that it does not return any data.
- It takes no parameters.
- When called this method will write the message "**hello**" to the **console**.

A method can be called using the **()** operator, as illustrated in the next code snippet:

```
nameOfTheFunction1();
nameOfTheFunction2(value);
int test = nameOfFunction3(value);
```

In the previous code, a method is called with no parameter (first line), or with a parameter (second line). On the third line, a variable called **test** will be set with the value returned by the method **nameOfFunction3** which takes one parameter.

Let's create our very own method:

- Please open the file **Program.cs**, and comment your previous code as in the next code snippet; to speed up things, and if you are working with Visual Studio, you can select these lines and then press simultaneously **CTRL + /** (or **APPLE + /**).

```
//Console.WriteLine("What is 1 + 1");
//string resultAsString = Console.ReadLine();
//int result = Int32.Parse(resultAsString);
//if (result == 2) Console.WriteLine("Well done");
//else Console.WriteLine("Sorry, wrong answer");
```

- Then add the following code to the **Main** method:

```
int myAge = calculateAge(1990);
Console.WriteLine("My Age is " + myAge);
}
```

In the previous code:

- We declare a variable called **myAge**.
- We call the method **calculateAge**, passing the number **1990** as a parameter.
- The information (or value) returned by the method is stored in the variable **myAge** and then displayed using the **alert** method.

We can now create the method called **calculateAge** by adding the following code after the method **Main**.

```
public static int calculateAge(int yearOfBirth)
{
return (2018 - yearOfBirth);
}
```

In the previous code:

- The method **calculateAge** is declared; it takes one parameter called **yearOfBirth**. So any parameter passed to this method will be referred as **yearOfBirth** within this method.
- This method returns the result of the subtraction **2018 – yearOfBirth**.
- When the method is called, **yearOfBirth** is equal to **1990**; so it will return **28** (i.e., **2018-1990)**.

Note that the new method called **calculateAge** is declared as static; this is because, the method main is static by default, and that as a result, it can only call a method that is also static. If you forget to declare this method as static and try to call it from the **Main** method, the system will generate a message saying **"An object reference is required for the non-static field, method, or property"**.

The Main method is static because it is the entry point of the programme, and it is the first method to be called (or invoked) when the application is started. It is **public** because it needs to be accessible from outside the application. Its return type is **void** because it usually doesn't return any data. It includes parameters but these are not compulsory.

For more information about the **Main** method, please see the official documentation here: **https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/main-and-command-args/**

Please save your code, and run the application, and you should see the following message in the console window.

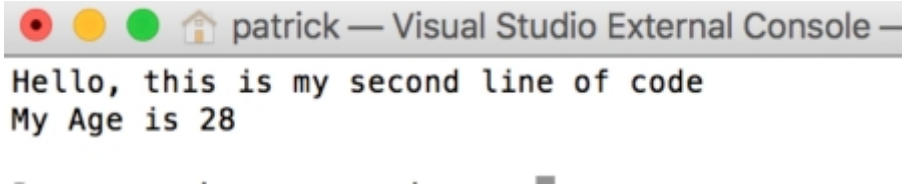

Figure 1-11: Displaying the value returned by the method

Scope of variables

Whenever you create variables in C#, you will need to be aware of their scope so that you use it where its scope makes it possible for you to do so. This is particularly important if you use methods, as some variables will be used just locally, whereas others will be used throughout the class or the application.

The scope of a variable refers to where you can use this variable in an application. In C#, in addition to access modifiers that will be covered later in this book, variables can be either **member variables** or **local variables**.

- Member variables can be used in a specific class or throughout your application, depending on their access type.

While we will study classes in detail; put simply, your application will be made of several parts called classes. In our case, the entry point for our application is a class called **MainClass**.

- Member variables can be declared at the start of the class, using the usual declaration syntax, preceded by what is called an access type (we will come back to this concept later on). They can then be used anywhere in the class (and sometimes outside the class also), as illustrated in the next code snippet.

```
class MainClass
{
public int myVar;
public void method1()
{
myVar = 0;
}
public void method2()
{
myVar = myVar + 1;
}
```

In the previous code, the member variable **myVar** is declared at the start of the class **MainClass**; it is then initialized in **method1**, and updated in **method2**. Note that the variable was declared as public, which means that not only is it accessible throughout the class called **MainClass**, but it is also accessible from outside this class.

- Local variables are declared (and should only be used) within a specific method; hence the term local, because they can only be used locally, as illustrated in the next code snippet.

```
public void method1()
{
int myVar;
myVar = 0;
}
public void method2()
{
int myVar2;
myVar2 = 2;
}
```

In the previous code, **myVar** is a local variable to the method **method**1, and can only be used within this method; **myVar2** is a local variable to the method **method**2, and can only be used within this method.

Note that it is always good to avoid what is called variable masking/shadowing, which occurs when you give a local variable the same name as a member variable in the same file, as this could lead to confusion.

Classes and member variables can be compared to a city with different clubs and societies. The city is made of these different clubs. Each club has members. Some of these members can only be accessed if you are already part of this specific club where they belong to; other members will be available more publically for people who are not part of the same club. Because some of these members (for example a treasurer) are very powerful and could change significantly the future and safety of the club, you may prefer that not everyone outside the club has access to them, as it would mean that anyone can ask them to make drastic changes to the club. However, all the club members, because they are trusted, could be granted access to this treasurer. On the other hand, each club could have a PR person in charge of promoting the club. In this case, this member could be made public so that anyone outside this club can access them to get information about the club (e.g., opening hours, features, etc.).

So let's experiment with variables and their scope:

- Please add the following code to the file **Program.cs** (new code in bold).

```
class MainClass
{
public static int myMemberVariable;
public static void myMethod()
{
myMemberVariable = 3;
Console.WriteLine("The value of myMemberVariable is " +
myMemberVariable);
}
```

```
public static void Main(string[] args)
{
myMemberVariable = 1;
Console.WriteLine("The value of myMemberVariable before calling the method is " + myMemberVariable);
myMethod();
```

In the previous code:

- We declare a member variable called **myMemberVariable**; because it is declared outside any method, this variable will be a member variable and accessible throughout our class.
- It is declared as static because it will be called from the **Main** method which is already static by default; and as we already know, a static method should only call a method that is also static.
- We then call the method **myMethod**.
- In the method called **myMethod**, we set and display the value of the variable **myMemberVariable**.
- Finally, when the application is launched, the **Main** method is called (as it is the point of entry of our application),
- We set the value of the variable **myMemberVariable** to **1**, and then call the method **myMethod**.

Please save your code and run your application. You should see a window similar to the following figure:

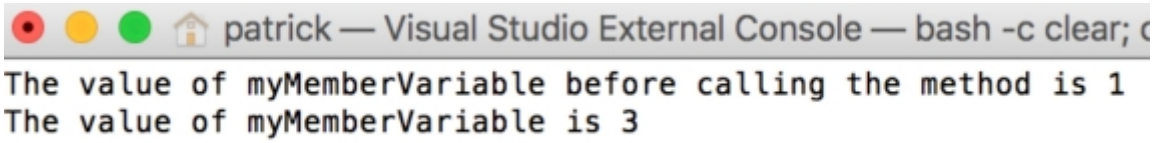

Figure 1-12: Using member variables

.

A few things to remember when you create your applications (checklist)

When you start coding, you will, as for any new activity, make small mistakes, learn what they are, improve your coding, and ultimately get better at writing your applications. As I have seen in the past with students learning programming, there are some common errors that are usually made; these don't make you a bad programmer; on the contrary, it is part of the learning process because ***we all learn by trial and error, and making mistakes is part of the learning process***.

So, as you create your first application, set any fear aside, try to experiment, be curious, and get to learn the language. It is like learning a new foreign language: when someone from a foreign country understands your first sentences, you feel so empowered! So, it will be the same with C#, and to ease the learning process, I have included a few tips and things to keep in mind when writing your applications, so that you progress even faster. You don't need to know all of these by now (I will refer to these later on, in the next chapter), but it is so that you are aware of it and also use this list if any error occurs. So, watch out for these: :-)

- Each opening bracket has a corresponding closing bracket.

- All variables are written consistently (e.g., spelling and case). The name of each variable is case-sensitive; this means that if you declare a variable **myvariable** but then refer to it as **myVariable** later on in the code, this may trigger an error, as the variable **myVariable** and **myvariable**, because they have a different case (upper-case **V**), are seen as two different variables.

- As much as possible, declare a variable prior to using it (e.g., **int myVar**).

- For each variable name, all words (except for the first word) have a capitalized initial. This is not a strict requirement, however, it will make your code more readable.

- For each method, all words (except for the first word) have a capitalized initial. This is not a strict requirement; however, it will make your code more readable.

- All statements are ended with a semi-colon.

- For **if** statements the condition is within round brackets.

- For **if** statements the condition uses the syntax "==" rather than "=".

- When calling a method, the exact name of this method (i.e., case-sensitive) is used.

- When referring to a variable, it is done with regards to the scope of the variable (e.g., call local variables locally).

- Local variables are declared and can be used within the same method.

- Member variables are declared outside methods and can be used anywhere within a class.

Best practices

As you will start your journey through C# coding, you may sometimes find it difficult to interpret the errors produced by the browser. However, after some practice, you will manage to recognize them, to understand (and also avoid) them, and to fix them accordingly. The next list identifies the errors that my students often come across when they start coding in C#.

Variable naming

- Use meaningful names that you can understand, especially after leaving your code for two weeks.

```
string myName = "Patrick";//GOOD
string b = "Patrick";//NOT SO GOOD
```

- Capitalize the initial of all words in the name, except for the first word.

```
bool testIfTheNameIsCorrect = true;// GOOD
bool testifthenameiscorrect = true; // NOT SO GOOD
```

Methods

- Use unique (i.e., different) names (for member and local variables).
- Check that all opening brackets have a corresponding closing bracket.
- Indent your code.
- Comment your code as much as possible to explain how it works.

Level roundup

Summary

In this chapter, we have managed to create your very first applications. You have become more comfortable with the creation of variables and methods, as well as the display of information through the console window. Finally, you also learned how to employ conditional statements so that processing can be performed based on specific conditions. Building on this knowledge, you created simple programs where the user enters information and where the program computes and outputs a result accordingly. So we have covered some significant ground here from no knowledge of C# to creating your first application: well done!

Quiz

It is now time to test your knowledge. Please specify whether the following statements are TRUE or FALSE. The answers are available on the next page.

1. The main point of entry in a C# program is the method called **Main**.
2. The **Main** method is usually **public** so that it can be called from outside your application.
3. The Main method is usually declared as **static**.
4. The command **Console.WriteLine** displays information in the *Console* window.
5. **Console.ReadLine** records information entered by the user in the *Console* window.
6. In C# text values are surrounded by double quotes.
7. In C# a local variable can only be used in the method where it was initially defined.
8. In C# a member variable can be used throughout the class where it was declared.
9. In C# a method always returns a value.
10. In C# a code that is commented is not executed.

Solutions to the Quiz

1. TRUE.
2. TRUE.
3. TRUE.
4. TRUE.
5. TRUE.
6. TRUE.
7. TRUE.
8. TRUE.
9. FALSE.
10. TRUE.

Checklist

You can move to the next chapter if you can do the following:

- Create variables.
- Combine variables.
- Create a method.
- Call a method.

The solutions for the following challenges are included in the **resource pack** that you can download by following the instructions included in the section entitled "**Support and Resources for this Book**".

Challenge 1

Now that you have managed to complete this chapter and that you have gathered interesting skills, let's put these to the test. This particular challenge will get you to become more comfortable with methods. The solutions are included in the resource pack.

Create a method called **calculatePrice** that does the following:

- Take two parameters called **nbItems** and **unitPrice**.
- Multiply these two numbers and store the result in a variable called **finalPrice**.
- Return the value of the variable **finalPrice**.

Once the method has been created, call this method using two values of your choice and display the result returned by the method in the console window.

Challenge 2

Create a method that does the following:

- Asks the user for his/her first name.
- Asks the user for his/her first last name.
- Displays the full name (i.e., first name followed by a space, followed by the last name) in the console window.
- Once the method has been created, call this method.

2
Thank you

I would like to thank you for completing this book. This book has only covered the basics of C# and learn more you need to download the book C# Programming from Zero to proficiency (Beginner) to learn even more C# programming skills using this link:

https://books2read.com/u/m2Mj9o

So that the book can be constantly improved, I would really appreciate your feedback. So, ***please leave me a helpful review*** letting me know what you thought of the book and also send me an email (**learncsharpforbeginners@gmail.com**) with any suggestions you may have. I read and reply to every email.

I have been publishing books for almost 6 years now, and while it has been a very exciting experience, it also demands a lot of energy and perseverance to manage to complete and publish these books. After publishing over 20 books on the topic of games and programming, I am delighted to have managed to help many people to get into coding, something that I had hoped but hardly imagined would happen when I first started publishing my books. Becoming a self-publishing author, has made it possible for me to publish quality, practical and helpful content that I know will benefit readers without being restricted by a publishing company in terms of content or format. This being said, as a self-published author, my work depends heavily on the reviews that I receive; without reviews, my books cannot get noticed, and will probably fall into oblivion; so if you have enjoyed this book and want to help people to find it and benefit from it, please leave a review on the ebook store of your choice; this would mean the world to me and also help to support my work.

Thanks so much!!

[ ]

Also by Patrick Felicia

Beginners' Guides
A Beginner's Guide to 2D Platform Games with Unity
A Beginner's Guide to 2D Shooter Games
A Beginner's Guide to Puzzle Games

C# from Zero to Proficiency
C# Programming from Zero to Proficiency (Introduction)
C# Programming from Zero to Proficiency (Beginner)

Getting Started
Getting Started with 3D Animation in Unity

Godot from Zero to Proficiency
Godot from Zero to Proficiency (Foundations)
Godot from Zero to Proficiency (Advanced)
Godot from Zero to Proficiency (Beginner)
Godot from Zero to Proficiency (Intermediate)
Godot from Zero to Proficiency (Proficient)

JavaScript from Zero to Proficiency
JavaScript from Zero to Proficiency (Beginner)

Learn Python by Coding Video Games
Learn Python by Coding Video Games (Intermediate)
Learn Python by Coding Video Games (Beginner)

Python Games From Zero to Proficiency
Python Games from Zero to Proficiency (Beginner)
Python Games from Zero to Proficiency (Intermediate)

Quick Guides
A Quick Guide to c# with Unity
A Quick Guide to Procedural Levels with Unity
A Quick Guide to 2d Infinite Runners with Unity
A Quick Guide to Artificial Intelligence with Unity

A Quick Guide to Card Games with Unity

Ultimate Guides
The Ultimate Guide to 2D games with Unity

Unity 5 from Proficiency to Mastery
Unity from Proficiency to Mastery (C# Programming)

Unity from Proficiency to Mastery
Unity from Proficiency to Mastery (Artificial Intelligence)

Unity from Zero to Proficiency
Unity from Zero to Proficiency (Foundations) Fifth Edition
Unity from Zero to Proficiency (Beginner)
Unity from Zero to Proficiency (Intermediate)
Unity from Zero to Proficiency (Advanced)
Unity from Zero to Proficiency (Proficient)

Unreal Engine from Zero to Proficiency
Unreal Engine from Zero to Proficiency (Foundations)
Unreal Engine From Zero to Proficiency (Beginner)

Standalone
Becoming Comfortable with Unity
Unity from Zero to Proficiency (Foundations): A Companion Book
Learn C# Programming by Creating Games with Unity
Master Procedural Generation with Unity and C#
Unity All-In-One

Watch for more at Patrick Felicia's site.